

提出日 : 2025 年 10 月 12 日

3D レンダリングの基本的な仕組み

学科 : 情報通信エレクトロニクス工学科

学籍番号 : 14 番

氏名 : 北御門翔一

1. 完成品

いちから C++言語で 3D レンダリング処理を実装した。以下のリンクから動画を見ることができる。映像では立方体と四面体が回転しているものである。

完成品の動画 <https://youtu.be/Usq6r2-WRY4>

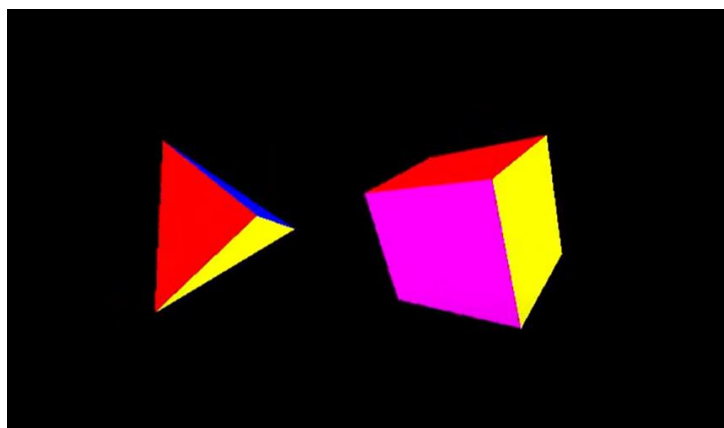


図 1 完成品のサムネイル

2. テーマと目的

ゲームなどで目にする 3DCG の仕組みを根本的に調査し、実際にプログラムを作成して 3D 描画処理を体験する。最近では、OpenGL, DirectX などの 3D 関係のライブラリが充実しており、だれでも簡単に 3D 描画処理を扱うことができる。そこで、ライブラリに頼らず根本的な仕組みづくりを体験し、3D への理解を深める。

3. 基本原理

3.1. 三角形 (triangle)

複雑な形状に見える 3D モデルも、その最小単位はすべて三角形である。三角形は 3 つの座標データをもち、三角形のデータ群がひとつの 3D モデルを形成する。例えば立方体は 12 個の三角形のデータを持っている。平面の正方形を形成するのに三角形が 2 つ必要であり、立方体は面が 6 つあるので、 2×6 で 12 個必要になるわけである。

図 1 は立方体を三角形で表したものである。立方体の中心を原点とした座標とその座標 3 つをセットとした三角形で構成されているのがわかる。プログラムの際には立方体を表すのに、8 つの座標データと 12 個の三角形データが必要になる。

また座標系は多くの 3D ソフトで使用されている右手座標系であり、今後も右手座標系で説明することにする。

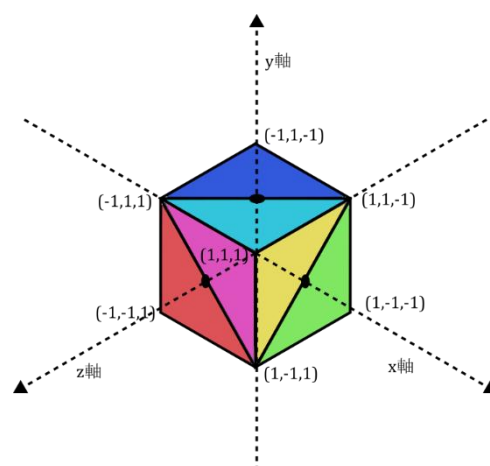


図 2 立方体を座標と三角形で表現

3.2. レンダリングの流れ

3D モデルを画面に表示するための工程をレンダリングといい、その工程はモデル変換、ビュー変換、投影変換、ビューポート変換、ラスタライズという流れで行っている。

基本的にレンダリングの最小単位は三角形であり、三角形ごとにレンダリングの工程を行う。

4. モデル変換 (model transformation)

前提として、3D モデルは三角形の集合でできている。三角形は 3 つの座標データで構成されている。この座標データはモデルを構成するための座標データで、**モデルのローカル座標**と呼ばれる。モデルには配置される座標データと回転データなどを含むため、それを使ってワールド上に配置しなければならない。このモデルのローカル座標をワールド空間に配置する変換をモデル変換という。ワールド座標とはレンダリングする世界の絶対的な座標であり、例えば世界地図の座標がワールド座標にあたる。

具体的なコードを示す。三角形ごとに処理を施すため、3 つの座標データをそれぞれ回転・平行移動している。

```
// 回転
Vector3 v0 = model->getRotation().Rotate(tri.v0);
Vector3 v1 = model->getRotation().Rotate(tri.v1);
Vector3 v2 = model->getRotation().Rotate(tri.v2);

// 平行移動
v0 = v0 + model->getPosition();
v1 = v1 + model->getPosition();
v2 = v2 + model->getPosition();
```

Rotate 関数は、3 次元の座標データを回転させるために、回転行列を用いることで三次元の座標データを変換している。例えば y 軸を回転させるためには、以下の行列を三次元ベクトルにかけることになる。変数 yaw は右手系で y 軸の回転を表すもので単位はラジアンである。

$$\begin{pmatrix} x' \\ y' \\ z' \end{pmatrix} = \begin{vmatrix} \cos yaw & 0 & \sin yaw \\ 0 & 1 & 0 \\ -\sin yaw & 0 & \cos yaw \end{vmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} \quad (1)$$

5. ビュー変換 (view transformation)

ビュー変換とは、ワールド座標系に配置された 3D モデルを、カメラを基準とした座標系に変換することである。カメラの向きベクトルを用いて 4×4 の行列を作成して、それを 4 次元に拡張した座標データにかけることで変換できる。4 次元に拡張しているのは回転と平行移動を同時に行うことができるためであり、3D 処理では、よく使われる処理方法である。

具体的なコードを示す。

```
// ビュー行列を取得
Matrix4 Camera::getViewMatrix() const {
    Vector3 zaxis = forward.normalize(); // カメラの向き
    Vector3 xaxis = up.cross(zaxis).normalize(); // 右方向
```

```

Vector3 yaxis = zaxis.cross(xaxis); // 上方向

Matrix4 view = Matrix4::identity();

view.m[0][0] = xaxis.x; view.m[1][0] = xaxis.y; view.m[2][0] = xaxis.z; view.m[3][0] = -xaxis.dot(position);
view.m[0][1] = yaxis.x; view.m[1][1] = yaxis.y; view.m[2][1] = yaxis.z; view.m[3][1] = -yaxis.dot(position);
view.m[0][2] = zaxis.x; view.m[1][2] = zaxis.y; view.m[2][2] = zaxis.z; view.m[3][2] = -zaxis.dot(position);

return view;
}

```

6. 投影変換 (projection transformation)

近くのものは大きく見え、遠くのものは小さく見えるように遠近感を出すための変換処理を投影変換という。これも 4×4 行列を作成して変換することができる。

具体的なコードを示す。

```

// 射影行列を取得
Matrix4 Camera::getProjectionMatrix() const {
    Matrix4 proj = {0};
    float f = 1.0f / tan(fov / 2);
    proj.m[0][0] = f / aspect;
    proj.m[1][1] = f;
    proj.m[2][2] = (farPlane + nearPlane) / (nearPlane - farPlane);
    proj.m[2][3] = -1.0f;
    proj.m[3][2] = (2 * farPlane * nearPlane) / (nearPlane - farPlane);
    return proj;
}

```

7. ビューポート変換 (viewport transformation)

これまでの変換を通してできた三次元座標データを実際の画面上の二次元座標データに変換することをビューポート変換という。またはスクリーン変換という。

具体的なコードを示す。

```

// 3D 座標をスクリーン座標に変換
Vector3 Camera::worldToScreen(const Vector3& worldPos) const {
    Matrix4 view = getViewMatrix();
    Matrix4 proj = getProjectionMatrix();
    Matrix4 vp = proj * view;

    Vector3 clipSpacePos = Matrix4::multiply(vp, worldPos);

    // NDC に変換
    Vector3 ndcPos = {
        clipSpacePos.x / clipSpacePos.z,

```

```

        clipSpacePos.y / clipSpacePos.z,
        clipSpacePos.z
    };

    // スクリーン座標に変換
    return {
        (ndcPos.x + 1) * 0.5f * screenWidth,
        (1 - (ndcPos.y + 1) * 0.5f) * screenHeight,
        ndcPos.z
    };
}

```

8. ラスタライズ (rasterization)

これまでの変換で、画面上のどこに三角形があるか二次元座標データを取得することができた。その三角形のあたりをピクセルごとに走査し、三角形の内側にあるなら色を塗るという処理をすることで、三角形の座標データから画面上のピクセルデータに変換できる。この処理をラスタライズという。

具体的なコードを示す。走査するピクセル範囲を決定するために、三角形を囲む最小矩形を $\text{minX} \sim \text{maxY}$ で求めている。三角形の三つのエッジを調べて、指定のピクセルが三つのエッジの内側にあれば色を塗るという処理である。

```

int minX = std::max(0, (int)std::floor(std::min({p0.x, p1.x, p2.x})));
int maxX = std::min(screenWidth - 1, (int)std::ceil(std::max({p0.x, p1.x, p2.x})));
int minY = std::max(0, (int)std::floor(std::min({p0.y, p1.y, p2.y})));
int maxY = std::min(screenHeight - 1, (int)std::ceil(std::max({p0.y, p1.y, p2.y})));

auto edge = [](int x0, int y0, int x1, int y1, int x, int y) {
    return (x - x0)*(y1 - y0) - (y - y0)*(x1 - x0);
};

float area = edge((int)p0.x, (int)p0.y, (int)p1.x, (int)p1.y, (int)p2.x, (int)p2.y);
if (area == 0) continue; // 面積0はスキップ

for (int y = minY; y <= maxY; ++y) {
    for (int x = minX; x <= maxX; ++x) {
        float w0 = edge((int)p1.x, (int)p1.y, (int)p2.x, (int)p2.y, x, y);
        float w1 = edge((int)p2.x, (int)p2.y, (int)p0.x, (int)p0.y, x, y);
        float w2 = edge((int)p0.x, (int)p0.y, (int)p1.x, (int)p1.y, x, y);

        if (w0 >= 0 && w1 >= 0 && w2 >= 0) { // 内部にある
            w0 /= area;
            w1 /= area;

```

```

w2 /= area;

// 深度補間
float z = 1.0f / (w0 / v0.z + w1 / v1.z + w2 / v2.z);

int idx = y * screenWidth + x;
if (z < zBuffer[idx]) {
    zBuffer[idx] = z;
    screenBuffer[idx] = tri.color;
}
}
}
}

```

9. Zバッファ

ラスタライズの際にZバッファという処理を行っている。これは画面にピクセルを記録する際に、そのピクセルを表示する三角形のその座標がカメラからどれくらい離れているのかを記録するものである。そうすることで、事前に描かれているピクセルとこれから描くピクセルのどちらがカメラに近いかが判断して上書きするべきかが判断できる。

これを実装することで以下の図3のように立方体と四面体が重なっても正しく描画される。

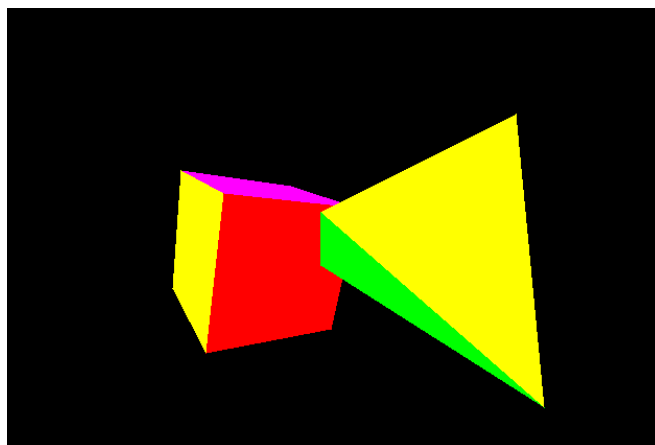


図3 Zバッファによる重なるの表現

10. 背面カリング (back-face culling)

そもそもカメラから見えない三角形は描画する必要はない。Zバッファの処理を実装していれば、正しく描画されることに違いはないが、処理にかかる時間が増える。なので、そもそもカメラの方向と三角形の面の方向が一緒であれば、裏面という判断になり描画をスキップでき、効率化できる。この処理を背面カリングという。

具体的なコードを示す。

```

// バックフェイスカリング
Vector3 u = v1 - v0;

```

```
Vector3 v = v2 - v0;  
Vector3 normal = (u ^ v).normalize();  
Vector3 viewDir = (camera->getPosition() - v0).normalize();  
if (normal * viewDir > 0) continue; // 裏面は描かない
```

11. ソースコード

このプロジェクトのソースコードは以下の Github リンクから全て見る事ができる. またソースコードの作成に一部生成 AI のサポートを受けている.

[ソースコードを保存している Github ページ](#)